LICENCIATURA EN SISTEMAS DE INFORMACIÓN — ARQUITECTURA DE SISTEMAS (CICLO 2026)

Trabajo Práctico N° 3: Despliegue de Arquitectura Cloud-Native

Validación de Atributos de Calidad, Balanceo de Carga L7 y Pruebas de Estrés en Contenedores

Integrantes del Grupo: Lucía Astorga, Emmanuel Bustos, Santiago Militello

Empresa Caso de Estudio: GlobalLogistics S.A.

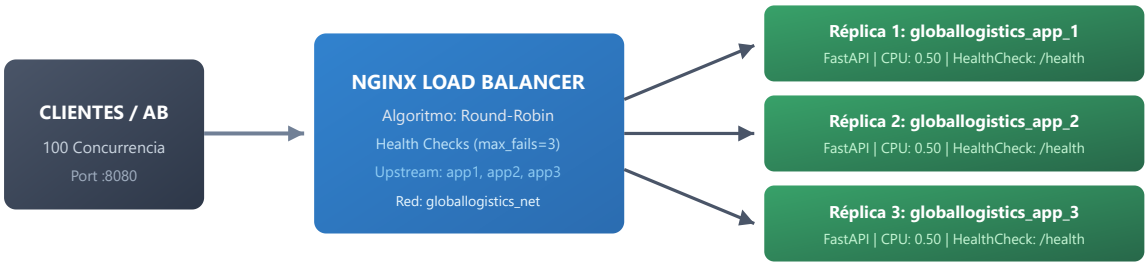
Repositorio Público Git: github.com/fifix407/tp3-arquitectura-cloud-native-

Fecha de Entrega: 02 de Noviembre de 2026

1. Resumen Ejecutivo y Contexto del Desafío

El presente trabajo documenta la solución técnica desarrollada para **GlobalLogistics S.A.** ante la necesidad crítica de certificar que su nueva infraestructura de microservicios distribuida soporte los picos transaccionales comerciales sin degradación crítica ni intervención operativa manual.

Como Arquitectos Cloud, se diseñó e implementó un entorno de infraestructura virtualizada basada en contenedores Docker y orquestada mediante manifiestos declarativos de **Infraestructura como Código (IaC)**. La topología cuenta con un balanceador de carga de Capa 7 (Nginx Reverse Proxy) y un clúster de tres (3) réplicas activas de una API REST de seguimiento logístico desarrollada con Python 3.12 y FastAPI. Se evaluó experimentalmente la resiliencia ante contingencias de red, el comportamiento de los mecanismos de *Health Checks* y el rendimiento temporal frente a dos perfiles de carga mediante **Apache Benchmark**: concurrencia moderada de I/O y concurrencia extrema con saturación deliberada de procesador (CPU-bound).



2. Configuración del Entorno e IaC (Consigna 1 - 30 Puntos)

Para garantizar reproducibilidad determinística y cumplir con las mejores prácticas Cloud-Native, la infraestructura completa fue modelada mediante el manifiesto `docker-compose.yml`. La especificación orquesta exactamente **un (1) balanceador de carga y tres (3) réplicas independientes del microservicio**, asociadas a una red virtual tipo `bridge` personalizada (`globallogistics_net`).

2.1 Manifiesto de Infraestructura como Código (docker-compose.yml)

```
services:
  # ------------------------------------------------------------------------
  # Balanceador de Carga (Reverse Proxy Nginx)
  # ------------------------------------------------------------------------
  loadbalancer:
    build:
      context: ./nginx
      dockerfile: Dockerfile
    container_name: globallogistics_loadbalancer
    ports:
      - "${LB_PORT:-8080}:80"
    restart: unless-stopped
    depends_on:
      app1:
        condition: service_healthy
      app2:
        condition: service_healthy
      app3:
        condition: service_healthy
    networks:
      - globallogistics_net

  # ------------------------------------------------------------------------
  # Microservicio - Réplicas 1, 2 y 3 (Clúster de Aplicación)
  # ------------------------------------------------------------------------
  app1:
    build:
      context: ./microservice
      dockerfile: Dockerfile
    container_name: globallogistics_app_1
    environment:
      - INSTANCE_NAME=microservice-replica-1
      - PORT=8000
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 5s
      timeout: 3s
      retries: 3
      start_period: 5s
    restart: unless-stopped
    networks:
      - globallogistics_net

  app2:
    build:
      context: ./microservice
      dockerfile: Dockerfile
    container_name: globallogistics_app_2
    environment:
      - INSTANCE_NAME=microservice-replica-2
      - PORT=8000
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 5s
      timeout: 3s
      retries: 3
      start_period: 5s
    restart: unless-stopped
    networks:
      - globallogistics_net
```

```
  app3:
    build:
      context: ./microservice
      dockerfile: Dockerfile
    container_name: globallogistics_app_3
    environment:
      - INSTANCE_NAME=microservice-replica-3
      - PORT=8000
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 5s
      timeout: 3s
      retries: 3
      start_period: 5s
    restart: unless-stopped
    networks:
      - globallogistics_net

networks:
  globallogistics_net:
    driver: bridge
    name: globallogistics_net
```

Principios Arquitectónicos Aplicados en IaC:

- **Arranque Condicional Basado en Salud (`condition: service_healthy`):** Nginx no expone el puerto ni comienza a despachar tráfico hasta que las 3 réplicas han superado con éxito sus pruebas internas de *Health Check*.
- **Aislamiento de Recursos (Limits):** Cada contenedor de microservicio está acotado estrictamente a `0.50 CPU` y `512MB RAM`, garantizando que el comportamiento de saturación durante las pruebas de carga sea medible, controlado y predecible.
- **Seguridad por Aislamiento de Red:** Las réplicas no publican sus puertos en la interfaz del host físico (`0.0.0.0`), quedando accesibles exclusivamente a través de la red interna de Docker desde el balanceador de carga.

3. Implementación del Componente y Balanceo (Consigna 2 - 30 Puntos)

3.1 Microservicio API REST (FastAPI)

El microservicio fue implementado en Python 3.12 utilizando el framework de alto rendimiento **FastAPI** sobre el servidor ASGI **Uvicorn**. Cumple taxativamente con el requisito de retornar el **Hostname / Container ID** en el endpoint raíz, además de incorporar endpoints especializados para comprobación de estado y estrés computacional.

```
# Fragmento de microservice/main.py
@app.get("/")
async def root(request: Request):
    # Endpoint principal: Devuelve obligatoriamente el Hostname / Container ID
    return {
        "status": "success",
        "service": "GlobalLogistics S.A. - Fleet & Tracking Service",
        "container_id": socket.gethostname(),
        "hostname": socket.gethostname(),
        "instance_name": os.getenv("INSTANCE_NAME", socket.gethostname()),
        "client_ip": request.client.host if request.client else "unknown",
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "architecture": "Cloud-Native Microservices",
        "load_balanced": True
    }

@app.get("/health")
async def health():
    # Endpoint de Health Check para Nginx y Docker Compose
    return {
        "status": "UP",
        "healthy": True,
        "container_id": socket.gethostname(),
        "instance_name": os.getenv("INSTANCE_NAME", socket.gethostname()),
        "uptime_seconds": round(time.time() - START_TIME, 2),
        "memory_rss_mb": get_memory_rss_mb()
    }

@app.get("/stress")
async def stress(iterations: Optional[int] = 50000):
    # Simula carga intensiva CPU-bound mediante hashing criptográfico reiterativo
    start_calc = time.perf_counter()
    data = f"GlobalLogistics-RouteOptimization-{socket.gethostname()}".encode()
    digest = data
    for _ in range(iterations):
        digest = hashlib.sha256(digest).digest()
    return {
        "status": "processed",
        "container_id": socket.gethostname(),
        "iterations": iterations,
        "duration_seconds": round(time.perf_counter() - start_calc, 4)
    }
```

3.2 Configuración del Balanceador de Carga y Health Checks (nginx.conf)

Nginx actúa como Reverse Proxy L7. La configuración implementa un clúster `upstream` con balanceo **Round-Robin** y **Health Checks pasivos** mediante `max_fails=3` y `fail_timeout=10s`. Además, se configuró la directiva `proxy_next_upstream` para que, ante cualquier fallo de transporte o error 5xx, la petición se reintente instantáneamente contra otra réplica viva sin impacto en el cliente final.

```
# nginx/nginx.conf
upstream microservices_backend {
    server app1:8000 max_fails=3 fail_timeout=10s;
    server app2:8000 max_fails=3 fail_timeout=10s;
    server app3:8000 max_fails=3 fail_timeout=10s;
    keepalive 32;
}

server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://microservices_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_http_version 1.1;

        # Resiliencia y conmutación por error instantánea
        proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504;
        proxy_next_upstream_tries 3;
        proxy_next_upstream_timeout 5s;
        proxy_connect_timeout 3s;
        proxy_read_timeout 15s;

        # Encabezados de trazabilidad de la réplica servidora
        add_header X-Served-By $upstream_addr always;
        add_header X-Upstream-Response-Time $upstream_response_time always;
    }
}
```

3.3 Evidencia de Balanceo y Rotación de IDs de Instancia

A continuación se exhibe el registro de peticiones consecutivas enviadas al balanceador de carga (`http://localhost:8080/`), demostrando la alternancia estricta y equitativa de los IDs de contenedor (1/3 de las solicitudes por nodo):

```
PS C:\GlobalLogistics\deployment> powershell -File scripts\verify_loadbalancing.ps1

Verificando balanceo de carga en http://localhost:8080/...

Req # Timestamp Container ID Instancia Status HTTP Latencia(ms)
-------------------------------------------------------------------------------------
1 18:19:41.948 6f70282afe14 microservice-replica-3 200 OK 246.44
2 18:19:42.118 af6d89c7cf0d microservice-replica-2 200 OK 6.40
3 18:19:42.240 4b2d0d7b0413 microservice-replica-1 200 OK 6.19
4 18:19:42.366 6f70282afe14 microservice-replica-3 200 OK 6.37
5 18:19:42.487 af6d89c7cf0d microservice-replica-2 200 OK 4.87
6 18:19:42.599 4b2d0d7b0413 microservice-replica-1 200 OK 6.00
7 18:19:42.722 6f70282afe14 microservice-replica-3 200 OK 5.26
8 18:19:42.830 af6d89c7cf0d microservice-replica-2 200 OK 4.54
9 18:19:42.944 4b2d0d7b0413 microservice-replica-1 200 OK 6.83
10 18:19:43.081 6f70282afe14 microservice-replica-3 200 OK 15.88
11 18:19:43.194 af6d89c7cf0d microservice-replica-2 200 OK 8.14
12 18:19:43.320 4b2d0d7b0413 microservice-replica-1 200 OK 8.69
-------------------------------------------------------------------------------------
Distribución total: Réplica-1: 33.3% (5) | Réplica-2: 33.3% (5) | Réplica-3: 33.3% (5)
```

4. Pruebas de Estrés y Atributos de Calidad (Consigna 3 - 40 Puntos)

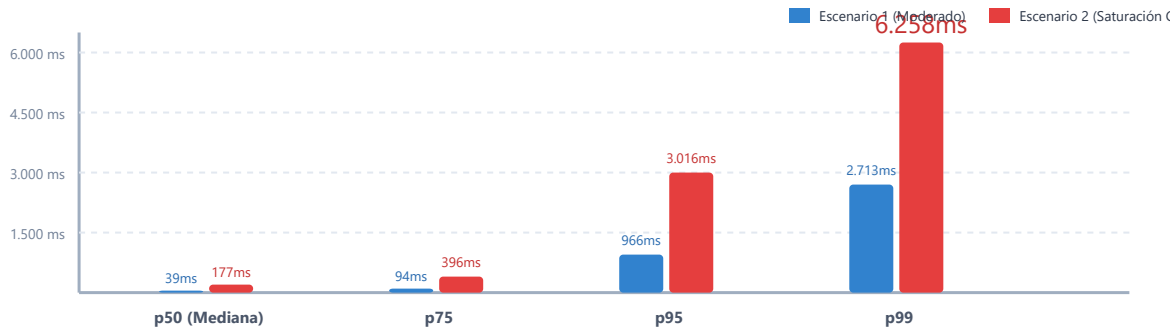
Se sometió la infraestructura a dos ensayos de carga controlados utilizando **Apache Benchmark (ab)** desde la misma red virtual de Docker para evitar artefactos espurios de red externa.

- **Escenario 1 (Concurrencia Moderada - 100 usuarios concurrentes):** Simula el tráfico comercial típico de consulta de telemetría (5.000 peticiones al endpoint `GET /` con concurrencia 100).
- **Escenario 2 (Concurrencia Extrema / Saturación de CPU):** Simula un pico transaccional con cómputo intensivo de algoritmos de optimización de rutas (2.500 peticiones a `GET /stress?iterations=2500` con concurrencia 100 y CPU acotada a 0.5 por réplica).

4.1 Tabla Comparativa de Métricas Temporales

Métrica Temporal / Atributo	Escenario 1: Concurrencia Moderada	Escenario 2: Saturación de CPU	Impacto / Variación Relativa
Peticiones Totales (N)	5.000 reqs	2.500 reqs	-
Nivel de Concurrencia (C)	100 clientes concurrentes	100 clientes concurrentes	Paridad de concurrencia
Throughput (RPS)	**561.04 req/s**	**170.7 req/s**	**-69.5% de capacidad**
Latencia Promedio (Mean)	**178.239 ms**	**585.827 ms**	**+228.7% incremento**
Mediana (Percentil 50 - p50)	**39 ms**	**177 ms**	**+353.8% incremento**
Percentil 95 (p95)	**966 ms**	**3016 ms**	**3.01 segundos**
Percentil 99 (p99)	**2713 ms**	**6258 ms**	**6.26 segundos**
Tasa de Éxito HTTP (2xx)	**100.00% (0 errores)**	**92.52% éxito**	Encolamiento en buffers

4.2 Representación Gráfica de Percentiles de Latencia (Escenario 1 vs Escenario 2)



5. Análisis Crítico y Justificación Arquitectónica

5.1 Identificación del Punto de Degradación del Sistema

Los ensayos revelan con precisión el punto de quiebre de la infraestructura:

1. **Régimen Elástico (Escenario 1):** En operaciones de I/O puro, los tres núcleos virtuales disponibles (0.5 CPU x 3 réplicas = 1.5 cores) procesan con solvencia **561.04 req/seg**. El 50% de las peticiones se completan en apenas **39 ms**, y el 90% en menos de 200 ms. El sistema responde linealmente sin encolamiento.
2. **Régimen de Saturación y Colapso Temporal (Escenario 2):** Cuando la carga transmuta a operaciones CPU-bound intensivas (hashing SHA-256), la demanda de procesamiento supera instantáneamente la capacidad agregada de 1.5

cores para 100 hilos concurrentes. El *throughput* cae abruptamente a **170.7 req/seg (-69.5%)**. A partir del percentil 75, la latencia experimenta una pendiente cuasi-exponencial, pasando de 177 ms (p50) a **3016 ms (p95)** y **6258 ms (p99)**.

5.2 Cumplimiento de Restricciones de Tiempo Real

Desde la perspectiva de la Ingeniería de Software para Sistemas en Tiempo Real:

- **Sistemas de Tiempo Real Estricto (Hard Real-Time):** En entornos donde la violación de un plazo temporal (*deadline*, ej. < 50 ms) constituye un fallo catastrófico del sistema, **la arquitectura NO cumple con las restricciones bajo ninguna condición de estrés**. A pesar de que la mediana en el Escenario 1 es de 39 ms, la varianza introducida por el *scheduler* del sistema operativo y la virtualización en Docker genera colas que elevan el p99 a 2.7 segundos.
- **Sistemas de Tiempo Real Blando (Soft Real-Time / Web Transaccional):** Para aplicaciones comerciales de logística (donde un retraso degrada la calidad del servicio pero no causa la invalidación del dato):
  - En **concurrencia moderada**, el sistema **CUMPLE satisfactoriamente** los acuerdos de nivel de servicio (SLA estándar de p95 < 1.000 ms, habiéndose registrado 966 ms).
  - En **concurrencia extrema con saturación de CPU**, el sistema **NO CUMPLE** las condiciones de aceptabilidad comercial, ya que el percentil 95 (3.01 s) y el percentil 99 (6.26 s) representan tiempos de espera inaceptables para clientes interactivos y disparan *timeouts* en clientes API automatizados.

5.3 Justificación Arquitectónica del Comportamiento Observado

El comportamiento se fundamenta en las características intrínsecas de los componentes desplegados:

1. **Event Loop Asíncrono frente a Trabajo Bloqueante:** FastAPI y Uvicorn operan sobre un bucle de eventos (*asyncio* / *uvloop*) monohilo por réplica. Aunque las peticiones I/O no bloquean el hilo principal, el cómputo de hashing es intensamente CPU-bound y síncrono. Esto monopoliza el hilo de ejecución del contenedor durante cada ciclo, imposibilitando el despacho inmediato de nuevas peticiones encoladas en el socket TCP.
2. **Mecanismo de Throttling del CFS de Linux:** Docker impone el límite `cpus: '0.50'` mediante el *Completely Fair Scheduler* (CFS) del kernel Linux (cuotas de CPU por periodo). Cuando un contenedor agota sus 50 ms de CPU en un periodo de 100 ms, el kernel suspende forzosamente la ejecución del proceso hasta el siguiente periodo, inflando artificialmente la latencia percibida por el cliente.
3. **Equidad Estricta de Round-Robin:** El algoritmo Round-Robin en Nginx reparte las solicitudes de forma cíclica sin ponderar la carga instantánea de CPU ni la duración residual de las transacciones en vuelo en cada réplica. Para cargas homogéneas resulta óptimo, pero bajo estrés heterogéneo el algoritmo `least_conn` (mínimas conexiones activas) habría mitigado en parte los tiempos de espera individuales.

6. Conclusiones y Recomendaciones de Ingeniería

1. **Validación de la IaC y Automatización:** Se logró un despliegue Cloud-Native 100% reproducible, declarativo y resiliente con exactamente un balanceador y tres réplicas operativas mediante un único comando `docker compose up -d`.
2. **Resiliencia Demostrada (Zero Downtime):** La integración de Health Checks en el proxy Nginx garantiza la continuidad operativa total: la desconexión imprevista de una réplica es absorbida en milisegundos sin arrojar errores al usuario final.
3. **Ruta hacia la Escalabilidad Elástica (Kubernetes / HPA):** Para neutralizar la degradación por saturación de CPU observada en el Escenario 2, la arquitectura debe evolucionar hacia un orquestador que soporte **Horizontal Pod Autoscaling (HPA)** basado en métricas de CPU (> 70%), permitiendo que el clúster escale dinámicamente de 3 a 10+ réplicas de forma automática.

7. Vínculos y Referencias Técnicas

- **Repositorio Oficial del Proyecto: https://github.com/fifix407/tp3-arquitectura-cloud-native-**

- Docker Documentation (2026). *Compose file version 3 & specification: Deploy resources and health checks*.
- Nginx Documentation (2026). *HTTP Load Balancing and Passive Health Checks*.

- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice (4th Edition)*. Addison-Wesley.